

Manual: Engenharia de Software

Ciclos de Vida, Requisitos, UML 2.5, SOLID, Padrões GoF, Microserviços, RESTful APIs, Testes de Software e CI/CD

CHANCELA EDITORIAL YLUNA85 LABS

Preparação Conjunta: Auditor Fiscal (TI) & Agente de Tributos Estaduais

Revisão Ortográfica e Glossário Geral de Siglas

Material de livre circulação interna. Revisado em Português Brasileiro (pt-BR).

1. Modelos de Ciclo de Vida e Processos de Desenvolvimento

A Engenharia de Software disciplina a aplicação de abordagens sistemáticas, estruturadas e quantificáveis ao desenvolvimento, operação e manutenção de software. O Ciclo de Vida compreende as fases pelas quais o sistema passa, desde a concepção inicial até o seu descarte.

- **Modelo Cascata (Waterfall / Clássico)**

Abordagem sequencial linear onde cada fase (Requisitos, Projeto, Codificação, Testes e Manutenção) deve ser 100% concluída antes de iniciar a próxima. Vantagem: alta previsibilidade e documentação rígida. Desvantagem: baixa flexibilidade a mudanças e entrega de software funcional apenas no final do ciclo.

- **Modelo Espiral (Barry Boehm)**

Modelo evolucionário e iterativo orientado à Análise e Gestão de Riscos. Cada ciclo da espiral é dividido em 4 quadrantes: 1. Determinação de Objetivos; 2. Avaliação e Redução de Riscos; 3. Desenvolvimento e Validação; 4. Planejamento da Próxima Fase.

- **Processo Unificado (RUP - Rational Unified Process)**

Processo iterativo e incremental centrado em arquitetura e guiado por Casos de Uso. Estruturado em 4 Fases temporais com marcos bem definidos: 1. Concepção (Inception - escopo e viabilidade do negócio); 2. Elaboração (Elaboration - arquitetura de referência e mitigação de riscos técnicos); 3. Construção (Construction - codificação e testes do produto); 4. Transição (Transition - homologação, implantação em produção e treinamento).

ALERTA DE PROVA - FASES DO RUP

A arquitetura de software é definida e a maior parte dos riscos técnicos é mitigada na fase de ELABORAÇÃO do RUP. Na fase de CONSTRUÇÃO ocorre o maior volume de codificação e geração de código!

2. Engenharia de Requisitos e Histórias de Usuário

A Engenharia de Requisitos compreende o conjunto de atividades voltadas para descobrir, analisar, documentar e manter os requisitos de um sistema computacional. Um requisito define o que o sistema deve fazer ou as restrições sob as quais ele deve operar.

- **Classificação dos Requisitos**

1. Requisitos Funcionais (RF): Descrevem as funções, serviços, comportamentos e transformações de dados que o software DEVE realizar (ex: 'O sistema deve calcular o ICMS devido'); 2. Requisitos Não Funcionais (RNF): Descrevem as restrições, propriedades emergentes e critérios de qualidade do sistema, classificados no modelo FURPS+ (Funcionalidade, Usabilidade, Confiabilidade/Reliability, Desempenho/Performance, Suportabilidade + Restrições de Design, Implementação, Interface e Físicas).

- **Processo de Engenharia de Requisitos**

Compreende 5 macroatividades interativas: 1. Elicitação / Levantamento (entrevistas, workshops, observação, prototipação); 2. Análise e Negociação (identificação de conflitos e inconsistências); 3. Especificação (documentação formal via SRS ou Casos de Uso); 4. Validação (revisões formais com clientes para garantir que o sistema atende às necessidades); 5. Gerenciamento de Requisitos (rastreabilidade e controle de mudanças).

- **Histórias de Usuário e o Critério INVEST**

Técnica ágil de especificação no formato: 'Como [persona], eu quero [ação], para que [benefício]'. Devem atender ao critério INVEST: Independent (independentes), Negotiable (negociáveis), Valuable (valiosas para o negócio), Estimable (estimáveis), Small (pequenas para caber em uma Sprint) e Testable (testáveis via critérios de aceite claros).

DIFERENÇA ENTRE RF E RNF NA PROVA

Requisito Funcional responde 'O QUE o sistema faz' (serviço ativo). Requisito Não Funcional responde 'COMO o sistema se comporta sob restrições de tempo, segurança, escalabilidade e plataforma'.

3. Modelagem de Sistemas com UML 2.5

A UML (Unified Modeling Language), mantida pelo OMG (Object Management Group), é a linguagem visual padrão da indústria para especificação, visualização, construção e documentação de artefatos de sistemas orientados a objetos. A UML 2.5 divide seus 14 diagramas em Estruturais e Comportamentais.

- **Diagramas Estruturais (Aspectos Estáticos)**

1. Diagrama de Classes (classes, atributos, métodos, visibilidades [+ público, - privado, # protegido, ~ pacote] e relacionamentos: Associação, Agregação [todo-parte fraco], Composição [todo-parte forte com ciclo de vida dependente], Generalização/Herança e Realização/Interface); 2. Diagrama de Componentes (módulos e dependências de software); 3. Diagrama de Implantação / Deployment (nós de hardware, servidores e artefatos físicos de execução); 4. Diagrama de Objetos; 5. Diagrama de Pacotes.

- **Diagramas Comportamentais (Aspectos Dinâmicos)**

1. Diagrama de Casos de Uso (atores, casos de uso e relacionamentos <<include>> [obrigatório/essencial] e <<extend>> [opcional/condicional sob ponto de extensão]); 2. Diagrama de Sequência (interação e troca de mensagens ordenadas no tempo entre linhas de vida); 3. Diagrama de Atividades (fluxo de controle e processamento paralelo com bifurcações/fork e junções/join); 4. Diagrama de Máquinas de Estados (transições de estados de um objeto disparadas por eventos).

ALERTA DE PROVA - RELACIONAMENTOS NO CASO DE USO

O relacionamento <<include>> indica execução OBRIGATÓRIA e mandatória do caso de uso incluído. O relacionamento <<extend>> indica execução OPCIONAL e condicional dependente de um ponto de extensão!

4. Princípios de Projeto Orientado a Objetos e SOLID

O design de software de alta qualidade busca alcançar Alto Acoplamento Coesivo e Baixo Acoplamento entre módulos. Os 5 princípios SOLID (formulados por Robert C. Martin - Uncle Bob) formam a base do desenvolvimento de software manutenível, extensível e testável.

- **Acoplamento versus Coesão**

Coesão: Grau em que as responsabilidades de uma classe ou módulo pertencem juntas (deve ser ALTA). Acoplamento: Grau de dependência e interconexão entre módulos diferentes (deve ser BAIXO/FRACO).

- **S - Single Responsibility Principle (Princípio da Responsabilidade Única)**

Uma classe deve ter um, e apenas um, motivo para mudar. Cada classe deve encapsular uma única responsabilidade funcional bem definida.

- **O - Open/Closed Principle (Princípio do Aberto/Fechado)**

Entidades de software (classes, módulos, funções) devem estar ABERTAS para extensão, mas FECHADAS para modificação. Novos comportamentos devem ser adicionados através de herança ou polimorfismo/interfaces, sem alterar o código-fonte original testado.

- **L - Liskov Substitution Principle (Princípio da Substituição de Liskov)**

Subtipos/classes derivadas devem ser capazes de substituir seus tipos base/superclasses sem que isso quebre a correteude e o comportamento esperado do programa.

- **I - Interface Segregation Principle (Princípio da Segregação de Interfaces)**

Muitas interfaces específicas para cada cliente são melhores do que uma única interface genérica e inflada. Nenhum cliente deve ser forçado a depender de métodos que não utiliza.

- **D - Dependency Inversion Principle (Princípio da Inversão de Dependência)**

Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de ABSTRAÇÕES. Abstrações não devem depender de detalhes; detalhes devem depender de abstrações (Injeção de Dependências).

MNEMÔNICO DOS PRINCÍPIOS SOLID

SOLID: Single Responsibility (uma função), Open/Closed (estender sem alterar), Liskov (subclasses substituem a base), Interface Segregation (interfaces enxutas) e Dependency Inversion (depende de abstrações/interfaces)!

5. Padrões de Projeto GoF (Gang of Four)

Padrões de Projeto (Design Patterns) são soluções gerais, reutilizáveis e consagradas para problemas recorrentes no desenvolvimento de software orientado a objetos, catalogados por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides (GoF) em 23 padrões divididos em 3 categorias.

- Padrões Criacionais (Criação de Objetos)

1. Singleton (garante que uma classe tenha uma única instância global no sistema); 2. Factory Method (define uma interface para criar um objeto, deixando as subclasses decidirem qual classe instanciar); 3. Abstract Factory (fornece uma interface para criar famílias de objetos relacionados sem especificar suas classes concretas); 4. Builder (separa a construção de um objeto complexo da sua representação); 5. Prototype (cria novos objetos clonando uma instância existente).

- Padrões Estruturais (Composição de Classes e Objetos)

1. Adapter (converte a interface de uma classe em outra interface esperada pelos clientes, permitindo que classes incompatíveis trabalhem juntas); 2. Facade (fornece uma interface simplificada e unificada para um subsistema complexo); 3. Decorator (agrega dinamicamente novas responsabilidades e comportamentos a um objeto em tempo de execução sem usar herança); 4. Composite (compõe objetos em estruturas de árvore para representar hierarquias todo-parte); 5. Proxy (fornece um substituto ou marcador de localização para controlar o acesso a outro objeto).

- Padrões Comportamentais (Comunicação entre Objetos)

1. Observer (define uma dependência um-para-muitos entre objetos para que, quando um mude de estado, todos os dependentes sejam notificados automaticamente - padrão Pub/Sub); 2. Strategy (define uma família de algoritmos encapsulados, tornando-os intercambiáveis em tempo de execução); 3. Template Method (define o esqueleto de um algoritmo em uma superclasse, postergando etapas para as subclasses); 4. Command (encapsula uma requisição como um objeto); 5. Chain of Responsibility (passa requisições ao longo de uma cadeia de manipuladores).

DIFERENÇA ENTRE ADAPTER, FACADE E DECORATOR

Adapter CONVERTE interfaces incompatíveis. Facade SIMPLIFICA o acesso a um subsistema complexo. Decorator ADICIONA novos comportamentos dinamicamente sem alterar a interface original!

6. Arquitetura de Software: Monolitos, Microsserviços e Clean Architecture

A arquitetura de software estabelece a estrutura organizacional fundamental de alto nível do sistema, seus componentes, relacionamentos e restrições de comunicação.

- **Monolito versus Arquitetura de Microsserviços**

Monolito: Aplicação única e autocontida onde todas as camadas e módulos compartilham a mesma base de código e banco de dados centralizado. Microsserviços: Abordagem arquitetural que estrutura o sistema como uma coleção de serviços autônomos, fracamente acoplados, com implantação independente (independent deployment), escalabilidade granular e bancos de dados descentralizados por domínio (Database-per-Service).

- **Padrões Arquiteturais em Microsserviços**

API Gateway (ponto único de entrada que roteia requisições, faz balanceamento e autenticação); Service Discovery (registro e localização dinâmica de instâncias de serviços); Circuit Breaker (interrompe chamadas a serviços com falha contínua para evitar efeito cascata); Saga Pattern (gerencia transações distribuídas entre microsserviços via orquestração ou coreografia de eventos).

- **Clean Architecture e Arquitetura Hexagonal (Ports & Adapters)**

Organização em camadas concêntricas onde a Regra de Dependência determina que as dependências de código-fonte devem apontar SEMPRE PARA O CENTRO (em direção às regras de negócio de alto nível). Centro: Entidades (Domain Entities) e Casos de Uso (Use Cases), totalmente isolados e desacoplados de frameworks, bancos de dados e interfaces de usuário.

A REGRA DE OURO DA CLEAN ARCHITECTURE

O núcleo de negócio (Entidades e Casos de Uso) NUNCA deve conhecer ou depender do banco de dados, da interface gráfica ou de bibliotecas externas. Os detalhes de infraestrutura é que dependem das abstrações centrais!

7. Web Services e APIs RESTful

APIs RESTful baseiam-se no estilo arquitetural REST (Representational State Transfer), introduzido por Roy Fielding, que utiliza o protocolo HTTP nativo para comunicação síncrona e interoperável entre sistemas distribuídos.

- **Os 6 Princípios Arquiteturais do REST**

1. Arquitetura Cliente-Servidor (separação de responsabilidades de UI e armazenamento); 2. Stateless (cada requisição do cliente deve conter TODAS as informações necessárias para ser processada, sem armazenar contexto de sessão no servidor); 3. Cacheable (as respostas devem ser explicitamente declaradas como passíveis ou não de cache); 4. Interface Uniforme (identificação de recursos por URIs, manipulação por representações JSON/XML e mensagens autoexplicativas); 5. Sistema em Camadas (a arquitetura pode ser composta por camadas intermediárias transparentes de proxy, gateway e segurança); 6. Código sob Demanda (opcional).

- **Métodos HTTP e Idempotência**

GET (recupera recurso; SEGURO e IDEMPOTENTE); POST (cria novo recurso; NÃO SEGURO e NÃO IDEMPOTENTE); PUT (substitui integralmente o recurso; NÃO SEGURO, mas IDEMPOTENTE); PATCH (atualização parcial do recurso; NÃO SEGURO e não necessariamente idempotente); DELETE (remove recurso; NÃO SEGURO, mas IDEMPOTENTE).

- **Códigos de Status HTTP Fundamentais**

1xx (Informativo); 2xx (Sucesso: 200 OK, 201 Created, 204 No Content); 3xx (Redirecionamento: 301 Moved Permanently, 304 Not Modified); 4xx (Erro do Cliente: 400 Bad Request, 401 Unauthorized [sem autenticação], 403 Forbidden [sem permissão], 404 Not Found); 5xx (Erro do Servidor: 500 Internal Server Error, 503 Service Unavailable).

CONCEITO DE IDEMPOTÊNCIA EM APIS

Uma operação HTTP é IDEMPOTENTE quando executá-la uma única vez ou múltiplas vezes consecutivas produz EXATAMENTE O MESMO resultado no servidor. São idempotentes: GET, PUT, DELETE e HEAD. O POST NÃO é idempotente!

8. Testes de Software e Garantia da Qualidade

O teste de software é o processo de execução de um programa com a intenção explícita de encontrar defeitos e verificar se o sistema atende aos requisitos especificados.

- **A Pirâmide de Testes (Martin Fowler)**

1. Testes Unitários / Unidade (base da pirâmide: alta quantidade, execução em milissegundos, baixo custo, testam classes e métodos isolados com Mocks); 2. Testes de Integração (meio: testam a comunicação entre múltiplos módulos, APIs e banco de dados); 3. Testes Ponta a Ponta / E2E e UI (topo: menor quantidade, execução mais lenta, testam o fluxo completo do usuário no navegador).

- **Testes Caixa-Preta (Funcionais)**

Testam o software sem conhecimento do código-fonte interno. Técnicas: Partição de Equivalência (divide as entradas em classes válidas e inválidas) e Análise do Valor Limite (testa os valores nas bordas e fronteiras das classes de equivalência, onde ocorre a maioria dos erros).

- **Testes Caixa-Branca (Estruturais)**

Testam a lógica interna, fluxos e estruturas do código. Técnicas: Cobertura de Instruções/Linhas, Cobertura de Ramos/Decisões e Teste de Caminhos Básicos.

- **Complexidade Ciclomática de McCabe (V(G))**

Métrica quantitativa de complexidade do código que indica o número de caminhos linearmente independentes no grafo de fluxo de controle: $V(G) = E - N + 2P$ (onde E é o número de arestas, N o número de nós e P o número de componentes conexos) ou $V(G) = \text{Nós Predicados (if, while, for)} + 1$.

- **Test-Driven Development (TDD)**

Ciclo Red-Green-Refactor: 1. Escreve um teste que falha (Red); 2. Escreve o código mínimo necessário para fazer o teste passar (Green); 3. Refatora o código para eliminar duplicações e melhorar o design mantendo o teste passando (Refactor).

CÁLCULO RÁPIDO DE COMPLEXIDADE CICLOMÁTICA

Em provas de concurso, a fórmula mais rápida para calcular a Complexidade Ciclomática de um trecho de código é:

$V(G) = (\text{Número de Condições / Nós de Decisão}) + 1$

9. DevOps, CI/CD e Controle de Versão com Git

A cultura DevOps integra desenvolvimento (Dev) e operações (Ops) por meio de automação contínua de entrega de software, testes automatizados e monitoramento de infraestrutura.

- **Controle de Versão Distribuído com Git**

Estrutura dos 3 Estados Locais: 1. Working Directory (arquivos em edição); 2. Staging Area / Index (arquivos preparados para commit via 'git add'); 3. Local Repository (histórico de commits salvos via 'git commit'). Comandos avançados: 'git rebase' (reescreve o histórico aplicando commits lineares), 'git merge' (cria commit de mesclagem), 'git cherry-pick' (aplica commit específico de outra branch) e 'git stash' (salva alterações temporárias em pilha sem comitar).

- **Pipelines de CI/CD (Integração e Entrega Contínua)**

Integração Contínua (CI): Desenvolvedores mesclam seu código frequentemente no repositório principal, disparando builds automáticos e suítes completas de testes unitários; Entrega Contínua (Continuous Delivery): O software aprovado no CI é empacotado e preparado automaticamente para implantação em produção mediante aprovação manual; Implantação Contínua (Continuous Deployment): Toda alteração aprovada no pipeline é implantada em PRODUÇÃO de forma 100% AUTOMÁTICA sem intervenção humana.

- **Estratégias Modernas de Deploy**

Blue-Green Deployment (mantém dois ambientes idênticos em produção, alternando o roteamento do roteador/balanceador); Canary Deployment (libera a nova versão para uma pequena porcentagem de usuários antes de liberar para 100% da base).

DIFERENÇA ENTRE CONTINUOUS DELIVERY E CONTINUOUS DEPLOYMENT

Continuous Delivery requer uma APROVAÇÃO MANUAL de um operador para subir em produção. Continuous Deployment sobe para produção de forma 100% AUTOMATIZADA assim que os testes passam!

10. As 10 Grandes Armadilhas de Engenharia de Software da FGV e CEBRASPE

Quadro analítico com as 10 principais armadilhas conceituais cobradas pelas bancas examinadoras em Engenharia de Software.

- **Pegadinha 1: POST NÃO é Idempotente**
GET, PUT e DELETE são idempotentes. Executar múltiplos POSTs cria múltiplos recursos duplicados no servidor.
- **Pegadinha 2: Singleton vs Factory Method**
Singleton garante instância única. Factory Method encapsula a lógica de instanciação deixando subclasses decidirem.
- **Pegadinha 3: Complexidade Ciclomática $SV(G) = P + 1$**
A complexidade ciclomática é a quantidade de predicados condicionais mais um, indicando o número mínimo de casos de teste.
- **Pegadinha 4: Clean Architecture Aponta para Dentro**
Todas as dependências de código apontam para o centro (regras de negócio). Entidades nunca conhecem o banco de dados.
- **Pegadinha 5: <<include>> é Obrigatório, <<extend>> é Opcional**
No Diagrama de Casos de Uso, include é mandatório e extend depende de condição específica.
- **Pegadinha 6: TDD é Red -> Green -> Refactor**
A ordem do TDD é: 1. Falha (Red); 2. Passa (Green); 3. Melhora (Refactor). Nunca se escreve o código antes do teste.
- **Pegadinha 7: Agregação vs Composição**
Composição é vínculo forte com destruição conjunta (ex: Pedido e Itens do Pedido). Agregação é vínculo fraco independente.
- **Pegadinha 8: Inversão de Dependência (DIP)**
Módulos de alto nível não dependem de baixo nível; ambos dependem de abstrações (interfaces).
- **Pegadinha 9: Rebase Lineariza o Histórico**
Git rebase reescreve o histórico para criar uma linha contínua, enquanto merge cria um commit de junção com dois pais.
- **Pegadinha 10: RUP Define Arquitetura na Elaboração**
A fase de Elaboração mitiga riscos e fecha a arquitetura de base; a Construção foca em codificação massiva.

SÍNTESE FINAL DE ENGENHARIA DE SOFTWARE

Lembre-se: POST não é idempotente; include é obrigatório; complexidade é predicados + 1; arquitetura limpa aponta para o centro; e no TDD o teste nasce vermelho!

GLOSSÁRIO DE SIGLAS E ABREVIATURAS

Consulte abaixo o significado e a expansão explicativa de todas as siglas contábeis, tributárias e de TI presentes nas apostilas de preparação:

ACID	Atomicidade, Consistência, Isolamento e Durabilidade. Propriedades fundamentais para transações em SGBDs.
BI	Business Intelligence. Inteligência de Negócios para mineração e análise estratégica de dados.
CAP	Consistência, Disponibilidade e Tolerância a Partições. Teorema limitador para sistemas distribuídos.
CEBRASPE	Centro de Seleção e de Promoção de Eventos. Banca examinadora oficial de concursos públicos.
CFC	Conselho Federal de Contabilidade. Órgão responsável pela edição das NBCs no território nacional.
CIAP	Controle de Crédito de ICMS do Ativo Permanente. Controle fiscal de apropriação de créditos de bens imobilizados.
CID	Confidencialidade, Integridade e Disponibilidade. Pilares de sustentação da Segurança da Informação.
COBIT	Control Objectives for Information and Related Technologies. Framework global de Governança de TI.
CPC	Comitê de Pronunciamentos Contábeis. Emissor de normas convergentes com padrões internacionais.
FGV	Fundação Getulio Vargas. Banca examinadora de certames e instituição de pesquisa econômica.
ICMS	Imposto sobre Operações Relativas à Circulação de Mercadorias e Prestações de Serviços de Transporte.
IPVA	Imposto sobre a Propriedade de Veículos Automotores. Tributo de competência estadual.
ITD	Imposto sobre Transmissão Causa Mortis e Doação. Tributo sobre heranças e doações na Bahia.
ITIL	Information Technology Infrastructure Library. Boas práticas de Gerenciamento de Serviços de TI.
KDD	Knowledge Discovery in Databases. Processo sistemático de descoberta de conhecimento em bases de dados.
LDO	Lei de Diretrizes Orçamentárias. Define as metas e prioridades para o orçamento do exercício seguinte.
LOA	Lei Orçamentária Anual. Estima as receitas e fixa as despesas públicas para o exercício financeiro.
NBC TA	Normas Brasileiras de Contabilidade Técnicas de Auditoria Independente. Alinhadas ao padrão internacional.
NBC TI	Normas Brasileiras de Contabilidade Técnicas de Auditoria Interna. Foco nos processos organizacionais.
NoSQL	Not Only SQL. Família de SGBDs não relacionais estruturados para alto desempenho e escalabilidade.
OLAP	Online Analytical Processing. Ferramentas analíticas baseadas em cubos de dados multidimensionais.
PAF-BA	Processo Administrativo Fiscal do Estado da Bahia. Lei nº 3.956/1981 que regula o contencioso.
PPA	Plano Plurianual. Planejamento estratégico governamental de médio prazo (4 anos).
RICMS-BA	Regulamento do ICMS do Estado da Bahia. Decreto regulamentador do principal imposto estadual.
RUP	Rational Unified Process. Processo de desenvolvimento de software disciplinado e preditivo.
SEFAZ-BA	Secretaria da Fazenda do Estado da Bahia. Órgão de arrecadação e controle fiscal baiano.

SGBD	Sistema de Gerenciamento de Banco de Dados. Software de controle e armazenamento seguro de dados.
SQL	Structured Query Language. Linguagem declarativa padrão para consulta em SGBDs relacionais.
TI	Tecnologia da Informação. Recursos de hardware, software e infraestrutura de rede corporativa.
XP	Extreme Programming. Metodologia ágil de engenharia de software focada em excelência técnica.